

Pythonプログラミング入門

目次

1. [はじめに](#)
2. [変数とデータ型](#)
3. [配列（リスト）](#)
4. [条件分岐](#)
5. [繰り返し処理](#)
6. [乱数の使い方](#)
7. [関数の作り方・使い方](#)
8. [2次元配列とボードゲーム](#)
9. [参考文献](#)

はじめに

Pythonは、初心者でも学びやすいプログラミング言語です。このテキストでは、Pythonの基本的な使い方を学んでいきましょう。

Pythonの特徴

- 読みやすい文法
- 豊富なライブラリ
- 様々な分野で使われている
- 無料で使える

変数とデータ型

変数は、データを保存する箱のようなものです。Pythonでは、以下のような基本的なデータ型があります。

数値型

```
# 整数 (int)
age = 10
print(age) # 出力: 10

# 小数 (float)
height = 1.5
print(height) # 出力: 1.5
```

文字列型

```
# 文字列 (str)
name = "たろう"
print(name) # 出力: たろう

# 文字列の連結
greeting = "こんにちは、" + name + "さん"
print(greeting) # 出力: こんにちは、たろうさん
```

真偽値型

```
# 真偽値 (bool)
is_student = True
is_adult = False
print(is_student) # 出力: True
```

配列（リスト）

配列は、複数のデータを順番に並べて保存できる箱です。

基本的な使い方

```
# リストの作成
fruits = ["りんご", "バナナ", "みかん"]
print(fruits) # 出力: ['りんご', 'バナナ', 'みかん']

# 要素の取得
print(fruits[0]) # 出力: りんご
print(fruits[1]) # 出力: バナナ

# 要素の追加
fruits.append("ぶどう")
print(fruits) # 出力: ['りんご', 'バナナ', 'みかん', 'ぶどう']

# リストの長さ
print(len(fruits)) # 出力: 4
```

条件分岐

条件分岐は、「もし～なら」という判断をプログラムで表現する方法です。

if文

```
# 基本的なif文
score = 80

if score >= 60:
    print("合格です!")
else:
    print("不合格です...")

# elifを使った条件分岐
age = 15

if age < 13:
    print("子供料金です")
elif age < 20:
    print("学生料金です")
else:
    print("大人料金です")
```

match文 (Python 3.10以降)

```
# match文の例
grade = "A"

match grade:
    case "A":
        print("優秀です!")
    case "B":
        print("良好です")
    case "C":
        print("普通です")
    case _:
        print("要努力です")
```

繰り返し処理

同じ処理を何度も繰り返すときに使います。

for文

```
# リストの要素を順番に処理
fruits = ["りんご", "バナナ", "みかん"]
for fruit in fruits:
    print(fruit)
```

```
# 範囲を指定した繰り返し
for i in range(5):
    print(i) # 0から4まで出力
```

while文

```
# 条件が真の間繰り返し
count = 0
while count < 5:
    print(count)
    count += 1
```

乱数の使い方

乱数は、ランダムな数を生成するときに使います。

```
import random

# 0から9までのランダムな整数
number = random.randint(0, 9)
print(number)

# リストからランダムに選ぶ
fruits = ["りんご", "バナナ", "みかん"]
random_fruit = random.choice(fruits)
print(random_fruit)

# 0から1までのランダムな小数
random_float = random.random()
print(random_float)
```

関数の作り方・使い方

関数は、特定の処理をまとめて名前をつけたものです。

基本的な関数

```
# 関数の定義
def greet(name):
    return "こんにちは、" + name + "さん"

# 関数の呼び出し
message = greet("たろう")
print(message) # 出力: こんにちは、たろうさん
```

引数と戻り値のある関数

```
def calculate_area(width, height):
    area = width * height
    return area

# 関数の使用
rectangle_area = calculate_area(5, 3)
print(f"長方形の面積は{rectangle_area}です") # 出力: 長方形の面積は15です
```

2次元配列とボードゲーム

2次元配列は、表のようなデータを表現するのに便利です。

オセロゲームの盤面を表現する例

```
# 8x8のオセロ盤を作成
# 0: 空きマス
# 1: 黒石
# 2: 白石
board = [[0 for _ in range(8)] for _ in range(8)]

# 初期配置
board[3][3] = 2 # 白石
board[3][4] = 1 # 黒石
board[4][3] = 1 # 黒石
board[4][4] = 2 # 白石

# 盤面の表示
def print_board(board):
    for row in board:
        for cell in row:
            if cell == 0:
                print(".", end=" ")
            elif cell == 1:
                print("●", end=" ")
            else:
                print("○", end=" ")
        print()

# 盤面を表示
print_board(board)
```

参考文献

1. Python公式ドキュメント: <https://docs.python.org/ja/3/>
2. いちばんやさしいPythonの教本 第2版 人気講師が教える基礎からサーバサイド開発まで
3. スッキリわかるPython入門
4. 独学プログラマー Python言語の基本から仕事のやり方まで